Operating Systems

A Complete Guide

Processes · Scheduling · Memory · File Systems · IPC · I/O · Security

Contents

What is an Operating System?

An **operating system (OS)** is the software layer that sits between applications and hardware. It manages resources, provides abstractions that hide hardware complexity, and enforces isolation between programs.

The layered model

User space	Browser, text editor, games, compilers, shell
System call interface	`open()`, `read()`, `write()`, `fork()`, `exec()`, `mmap()`
Kernel	Process management, memory management, file system, network stack, device drivers
Hardware	CPU, RAM, disk, NIC, GPU, USB, display

Core OS roles

- **Resource manager.** Decides who gets CPU time, memory, and I/O access — fairly and efficiently.
- **Abstraction provider.** Turns complex hardware into simple APIs. A "file" hides disk sectors; a "socket" hides network packets.
- **Isolation enforcer.** Ensures one process cannot read or corrupt another's memory. Crashes stay contained.
- **Security gatekeeper.** Controls permissions, users, and capabilities. Prevents unauthorized access to resources.

Processes & Threads

A **process** is an instance of a running program — it has its own memory space, file handles, and execution state. A **thread** is a lightweight unit of execution within a process, sharing memory with its siblings.

Process lifecycle

New $\xrightarrow{\text{admitted}}$ **Ready** $\xrightarrow{\text{scheduled}}$ **Running** $\xrightarrow{\text{exit}}$ **Terminated**

- Running → Ready: preempted by the scheduler
- Running → Waiting: process makes an I/O request
- Waiting → Ready: I/O operation completes

Process Control Block (PCB)

The kernel stores each process's metadata in a PCB:

- Process ID (PID), parent PID
- Current state (new, ready, running, waiting, terminated)
- CPU register values (saved on context switch)
- Memory maps (page table base pointer)
- Open file descriptors
- Scheduling priority and CPU time accounting

Memory layout of a process

Kernel space	Reserved; inaccessible from user space
Stack (grows ↓)	Local variables, return addresses, function frames
Guard page	Unmapped — catches stack overflow
Heap (grows ↑)	Dynamic allocations via `malloc`/`new`
BSS + Data	Uninitialised and initialised global/static variables
Text (code)	Read-only compiled instructions; lowest address

fork() and exec()

Unix systems create new processes with two system calls:

- `fork()` — duplicates the calling process. The child is an exact copy (copy-on-write pages, same open files). Returns 0 in the child, the child's PID in the parent.
- `exec()` — replaces the current process image with a new program. The shell uses `fork()` then `exec()` for every command you type.

Process vs thread

	Process	Thread
Address space	Private	Shared within process
Communication	IPC (pipes, sockets, ...)	Direct memory access
Isolation	Strong	Weak (a crash can kill siblings)
Creation cost	High	Low
Context switch	Expensive (TLB flush)	Cheaper

CPU Scheduling

The **scheduler** decides which ready process runs next on the CPU. Goals: maximise CPU utilisation, minimise average wait time, be fair, and meet deadlines.

Key metrics

- **CPU utilisation** — fraction of time the CPU is busy.
- **Throughput** — number of processes completed per unit time.
- **Turnaround time** — total time from submission to completion.
- **Waiting time** — time spent in the ready queue.
- **Response time** — time from submission to first response.

Scheduling algorithms

FCFS — First Come First Served

Processes run in arrival order. Simple to implement, but a long job can block all shorter jobs behind it (*convoy effect*).

```
Queue: P1(4ms)  P2(2ms)  P3(3ms)
Timeline: [P1 4ms][P2 2ms][P3 3ms]
Wait: P1=0, P2=4, P3=6  ->  avg 3.3 ms
```

SJF — Shortest Job First

Optimal average wait time (provably), but requires knowing burst time in advance. Long jobs risk starvation.

```
Sorted: P2(2ms)  P3(3ms)  P1(4ms)
Timeline: [P2 2ms][P3 3ms][P1 4ms]
Wait: P2=0, P3=2, P1=5  ->  avg 2.3 ms
```

Round Robin (RR)

Each process gets a fixed *time quantum* q. After q ms it is preempted and the next process runs. Excellent for interactive systems.

Trade-off

Smaller quantum ⇒ better response time but more context switches.
Larger quantum ⇒ approaches FCFS behaviour.

Priority scheduling

Each process is assigned a numerical priority; the highest priority runs first. Risk: low-priority processes starve. Mitigation: **aging** — gradually increase the priority of waiting processes.

Multilevel Feedback Queue (MLFQ)

The industry standard (Linux, macOS, Windows). Key rules:

1. New jobs enter the highest-priority queue (short quantum).
2. If a job uses its full quantum, it drops to a lower queue.
3. I/O-bound and interactive jobs use little CPU, so they stay at the top.
4. Periodic priority boost returns all jobs to the top queue to prevent starvation.

Q0	2 ms quantum	Interactive, I/O-bound tasks
Q1	4 ms quantum	Mixed workloads
Q2	8 ms quantum	CPU-bound batch jobs

Memory Management

The OS gives each process the illusion of its own large private address space (*virtual memory*), while managing physical RAM efficiently through paging and swapping.

Virtual memory and paging

- The address space is divided into fixed-size **pages** (typically 4 KB).
- Physical RAM is divided into same-sized **frames**.
- The kernel maintains a **page table** mapping virtual pages to physical frames.
- The **MMU** (Memory Management Unit) performs translation on every memory access.

Address translation

```
Virtual address 0x0000_4A20
  -> page number 0x4A, offset 0x20
  -> page table: page 0x4A -> frame 0x1C
  -> physical address: frame 0x1C, offset 0x20
```

TLB — Translation Lookaside Buffer

A small, fast hardware cache for page table entries. On a TLB *hit*, translation takes a single cycle. On a *miss*, the page table must be walked (several memory accesses). A context switch flushes or tags the TLB.

Demand paging and page faults

Pages are loaded into RAM only when first accessed (*demand paging*). Accessing an unmapped page triggers a **page fault**:

1. Hardware raises a page fault exception.
2. OS checks: is the address valid? Is the page on disk (swap)?
3. OS finds a free frame (evicting one if necessary).
4. OS reads the page from disk into the frame.
5. OS updates the page table and resumes the faulting instruction.

Page replacement algorithms

When RAM is full, the OS must evict a page. Choosing the wrong one is costly (disk I/O).

Algorithm	Policy	Note
OPT	Evict the page used furthest in the future	Optimal but impractical
LRU	Evict the least recently used page	Great hit rate; costly to track
CLOCK	Approximate LRU with a reference bit	Used in Linux
FIFO	Evict the oldest loaded page	Simple; poor performance

Storage hierarchy

Level	Latency	Capacity	Location
Registers	$<$1 ns	Bytes	Inside CPU
L1/L2/L3 cache	1–40 ns	KB–MB	On-chip
RAM (DRAM)	$\sim$100 ns	GB	Main memory
SSD (NVMe)	$\sim$100 µs	TB	Persistent
HDD	$\sim$10 ms	TB	Persistent
Tape / Archive	Seconds	PB	Off-site

File Systems

A file system organises data on disk into a hierarchy of files and directories, mapping human-readable names to physical disk blocks.

Inodes

Every file is described by an **inode** (index node) containing:

- File size, type (regular, directory, symlink, ...)
- Owner UID, group GID, permissions
- Timestamps: *ctime* (metadata change), *mtime* (content modified), *atime* (last access)
- Pointers to data blocks (direct, indirect, double-indirect)

Directories are files that map filenames to inode numbers.

Journaling

To survive crashes, modern file systems write changes to a **journal** (log) before applying them to the main data structures. On recovery, the OS replays or discards incomplete journal entries.

- **ext4** — standard Linux file system; journaling of metadata and optionally data.
- **NTFS** — Windows; transactional journal.
- **APFS** — macOS; copy-on-write, snapshots, strong encryption.
- **ZFS** — enterprise; always-on checksums, RAID-Z, copy-on-write.

Hard links vs symbolic links

	Hard link	Symbolic (soft) link
What it is	Another directory entry pointing to the same inode	A file containing a path string
Cross-filesystem	No	Yes
Target deleted	File survives (ref count > 0)	Link becomes dangling

VFS — Virtual File System

The VFS is a kernel abstraction layer. Application code calls `read()` identically whether the file lives on ext4, NFS, NTFS, or a FUSE mount. VFS dispatches to the correct driver.

Unix permission model

```
-rwxr-xr--  1  alice  staff  4096  notes.txt
|\_/\_/\_/
| owner  group  others
|
file type (- = regular, d = directory, l = symlink)

Permissions: r=4, w=2, x=1
chmod 754 notes.txt  <=>  -rwxr-xr--
```

IPC & Synchronisation

Processes must communicate and coordinate access to shared resources. Without synchronisation, shared data leads to race conditions.

IPC mechanisms

Mechanism	Description	Speed
Pipe	One-way byte stream; `ls | grep txt`	Fast (kernel buffer)
Named pipe (FIFO)	Pipe with a filesystem name; unrelated processes	Fast
Shared memory	Processes map the same physical pages	Fastest
Message queue	Kernel-managed typed message buffer	Medium
Socket (Unix)	Bidirectional; local only	Fast
Socket (TCP/UDP)	Bidirectional; across machines	Network speed
Signal	Asynchronous notification; very limited data	Instant

Race conditions and critical sections

A **race condition** occurs when the outcome of a computation depends on the relative timing of operations in concurrent threads or processes. The **critical section** is the code that accesses shared state and must not be executed by more than one thread at a time.

Requirements for a correct solution (Dijkstra 1965):

1. **Mutual exclusion** — only one process in the critical section at once.
2. **Progress** — if no process is in the section, a waiting process must eventually enter.
3. **Bounded waiting** — no process waits forever.

Synchronisation primitives

Mutex Binary lock. Holder has exclusive access. Others block (sleep) until released.

Semaphore Counter-based. `wait()` decrements (blocks if 0); `signal()` increments and wakes a waiter. A binary semaphore behaves like a mutex.

Spinlock Busy-waits (polls in a loop) instead of sleeping. Efficient for very short critical sections on multi-core systems. Wastes CPU if the wait is long.

Condition variable Used with a mutex to allow threads to wait for an arbitrary condition (`wait`, `signal`, `broadcast`).

Read-write lock Allows concurrent reads; exclusive writes.

Deadlock

Coffman's four necessary conditions (1971)

All four must hold simultaneously for deadlock to occur. Breaking any one prevents deadlock.

1. **Mutual exclusion** — at least one resource held exclusively.
2. **Hold and wait** — a process holds resources while waiting for more.
3. **No preemption** — resources cannot be taken away forcibly.
4. **Circular wait** — P_1 waits for P_2, P_2 waits for P_1 (or a longer cycle).

Strategies for handling deadlock:

- **Prevention** — eliminate one of the four conditions by design.
- **Avoidance** — use Banker's Algorithm to refuse requests that would lead to an unsafe state.
- **Detection and recovery** — allow deadlocks; periodically detect and break them (kill or roll back a process).
- **Ostrich algorithm** — ignore the problem (acceptable when deadlocks are extremely rare).

I/O & Storage

I/O is the bottleneck of most systems. The OS uses interrupts, DMA, and caching to keep the CPU busy while slow devices work.

I/O methods

Polling (busy-wait) CPU repeatedly reads a status register until the device is ready. Simple but wastes CPU cycles.

Interrupt-driven Device signals the CPU via an interrupt when done. CPU can do other work in the meantime. Hardware interrupt → ISR (Interrupt Service Routine) runs.

DMA (Direct Memory Access) A dedicated DMA controller moves data between device and RAM without CPU involvement. CPU only sets up the transfer and handles the completion interrupt.

Device types

Type	Description	Examples
Block device	Random access in fixed-size blocks	HDD, SSD, USB flash
Character device	Sequential byte stream	Keyboard, serial port, TTY
Network device	Packets, not bytes	Ethernet NIC, WiFi

All are exposed as files under `/dev/` on Unix systems.

Disk scheduling (HDD)

Mechanical disks have a slow seek step (moving the read/write head). The OS reorders pending requests to minimise total head movement:

- **SSTF** (Shortest Seek Time First) — serves the closest request next. Risk: starvation of far-away requests.
- **SCAN (elevator)** — head sweeps back and forth, serving requests in each direction.
- **C-SCAN** — only serves requests in one direction; resets to the start. More uniform wait times.

SSDs

Solid-state drives have no mechanical parts. Seek time is irrelevant. NVMe SSDs use a parallel command queue (up to 65,535 queues × 65,535 commands), making SCAN-style scheduling unnecessary.

The buffer/page cache

The kernel caches recently read disk blocks in RAM (the *page cache*). Writes are buffered (*write-back*) and flushed periodically by `pdflush`/`kworker`. `fsync()` forces an immediate flush to disk.

OS Security

The OS enforces security through isolation, access control, and privilege separation. Every system call is a potential security boundary.

Privilege rings

x86 processors define four privilege rings (0–3):

- **Ring 0** — kernel mode. Full hardware access.
- **Ring 3** — user mode. No direct hardware access. Restricted instruction set.

Rings 1 and 2 exist but are unused by mainstream OSes. A system call is the controlled gateway from ring 3 to ring 0: the CPU switches mode, validates arguments, executes the kernel function, and returns to ring 3.

Access control models

DAC — Discretionary Access Control Owners set permissions on their own objects (Unix `chmod`, Windows ACLs). Flexible but users can make mistakes.

MAC — Mandatory Access Control System-wide policy enforced by the OS regardless of user wishes. SELinux, AppArmor, and iOS sandbox use MAC. MAC overrides DAC.

RBAC — Role-Based Access Control Permissions assigned to roles; users assigned to roles. Common in enterprise systems and databases.

Security techniques

ASLR *Address Space Layout Randomisation*. Randomises the load addresses of stack, heap, and libraries. Makes buffer-overflow exploits much harder.

Stack canaries A random sentinel value placed before the return address. Checked before a function returns — if corrupted, the OS kills the process.

NX/XD bit Marks memory pages as non-executable. Prevents shellcode injected into the data or stack from running.

Capabilities Fine-grained privileges (`CAP_NET_BIND_SERVICE`, `CAP_SYS_PTRACE`, ...). A process can hold specific capabilities without being fully root.

Seccomp Filters which system calls a process is allowed to make. Chrome and Docker use seccomp to sandbox untrusted code.

Namespaces and cgroups (containers)

Linux containers (Docker, LXC, Podman) rely on two kernel features:

- **Namespaces** — isolate a process's view of the system: PID, network, mount, UTS, IPC, user,

cgroup namespaces.
- **cgroups** (control groups) — limit and account for CPU, memory, I/O, and network usage per group of processes.

Together they provide strong isolation without a full hypervisor.

Types of Operating Systems

OSes are designed for different trade-offs: real-time response, throughput, interactivity, scalability, or simplicity.

By kernel architecture

Architecture	Design	Examples
Monolithic	All kernel services in one address space. Fast via direct function calls. A bug can crash everything.	Linux, traditional Unix
Microkernel	Only IPC, memory, and scheduling in ring 0. All else in user-space servers. More fault-tolerant; IPC overhead.	QNX, seL4, Minix
Hybrid	Monolithic core with some subsystems in user space.	macOS (XNU), Windows NT
Exokernel	Kernel only multiplexes hardware; all abstraction in user-space libraries. Maximum flexibility.	Research (MIT Exokernel)
Unikernel	Application + minimal OS compiled into a single image. Tiny attack surface.	MirageOS, IncludeOS

By deployment context

General-purpose Windows, macOS, Linux. Balance interactivity, multitasking, security, and hardware compatibility.

Real-time (RTOS) FreeRTOS, VxWorks, QNX. Hard deadlines — a missed interrupt is a system failure. Used in pacemakers, automotive ECUs, factory robots, and avionics.

Embedded Runs on microcontrollers (Arduino, ESP32). Tiny footprint, often no MMU or file system. Power efficiency is paramount.

Distributed Manages a cluster as one logical machine. Google's Borg/Kubernetes, Plan 9. Modern cloud infrastructure approximates this model.

Mobile Android (Linux kernel + ART runtime), iOS (XNU kernel + sandboxing). Optimised for battery life and touch interaction.

Hard vs soft real-time

	Hard real-time	Soft real-time
Missed deadline	Catastrophic failure	Degraded quality
Examples	Airbag controller, pacemaker	Video streaming, audio playback
Scheduling	Rate-monotonic, EDF	Best-effort with priorities

Quick Reference

Key system calls (Linux)

Call	Category	Purpose
`fork()`	Process	Clone current process
`exec()`	Process	Replace process image
`wait()`	Process	Wait for child to exit
`open()`	File	Open or create a file
`read()` / `write()`	File	Read/write file data
`mmap()`	Memory	Map file/device into address space
`brk()`	Memory	Adjust heap end pointer
`pipe()`	IPC	Create a pipe
`socket()`	IPC/Net	Create a socket endpoint
`kill()`	Signal	Send signal to a process
`ioctl()`	Device	Device-specific control

Glossary

Context switch
Saving a running process's state and restoring another's. A fundamental OS overhead.

Thrashing
When the OS spends more time swapping pages than doing useful work. Caused by too many processes competing for too little RAM.

Interrupt
Hardware signal to the CPU that suspends current execution and invokes an ISR.

System call
The controlled mechanism by which user-space code requests a kernel service.

Zombie process
A process that has exited but whose PCB hasn't yet been collected by its parent (`wait()`).

Orphan process
A process whose parent exited first; adopted by `init`/`systemd` (PID 1).

Starvation
A process never gets CPU time because higher-priority processes always preempt it.

Aging
Gradually increasing a process's priority the longer it waits, to prevent starvation.

Thrashing
Excessive paging that degrades system performance.

Preemption
The OS forcibly removing the CPU from a running process.

Re-entrant
A function safe to call from multiple threads simultaneously.

POSIX
Portable Operating System Interface — a standard for Unix-like OS APIs.

For further reading: *Operating System Concepts* (Silberschatz et al.), *Modern Operating Systems* (Tanenbaum), *Three Easy Pieces* (Arpaci-Dusseau) — freely available at `ostep.org`